LABORATORIO

Lenguaje C

Fundamentos de Programación

Modalidad: Grupal

Entrega: Un único archivo `.zip` o `.rar` con ambas partes separadas en carpetas.

Requisito de aprobación: Es obligatorio presentar todos los laboratorios del curso.

0. Estructura de Entrega

El archivo comprimido debe respetar exactamente la siguiente estructura de directorios:

```
entrega_<grupo>/
├── primera_parte/
│   ├── global.h
│   ├── global.c          (implementación)
│   └── main.c            (programa de prueba)
└── segunda_parte/
    ├── estudiantes.h
    ├── estudiantes.c
    └── main.c
```

⚠ **Importante:** El incumplimiento de la estructura de carpetas o los nombres de archivos implicará la corrección del laboratorio como incorrecto en su totalidad.

Uso de Git (opcional pero recomendado): Se valorará positivamente la entrega del historial de commits que refleje el progreso del trabajo.

1. Configuración del Entorno de Desarrollo

Para compilar y ejecutar los programas de este laboratorio en Windows se requiere instalar las siguientes herramientas. Se recomienda seguir el orden indicado.

1.1 GCC (MinGW-w64)

GCC es el compilador de C que se utilizará durante todo el laboratorio.

1. Descargar el instalador desde: https://www.mingw-w64.org/downloads/
2. Seleccionar la arquitectura x86_64 y el hilo posix durante la instalación.
3. Agregar la carpeta bin de MinGW al PATH del sistema (ej.: C:\mingw64\bin).
4. Verificar la instalación abriendo una terminal y ejecutando:

```
gcc --version
```

1.2 CMake

CMake permite gestionar el proceso de compilación de forma portable.

5. Descargar el instalador desde: https://cmake.org/download/
6. Durante la instalación, marcar la opción "Add CMake to the system PATH".
7. Verificar con:

```
cmake --version
```

1.3 Python 3

Python es requerido por algunas herramientas auxiliares de CMake y scripts de automatización.

8. Descargar desde: https://www.python.org/downloads/
9. Marcar "Add Python to PATH" durante la instalación.
10. Verificar con:

```
python --version
```

1.4 Visual Studio Code + Extensiones

VS Code es el editor recomendado. Una vez instalado, agregar las siguientes extensiones:

Extensión	Propósito
`C/C++ (Microsoft)`	Autocompletado, navegación y depuración de código C/C++.
`CMake Tools`	Integración de CMake directamente desde el editor.
`Code Runner`	Compilar y ejecutar programas con un solo clic.
`clangd (opcional)`	Análisis estático avanzado y sugerencias de código.

Para compilar manualmente desde la terminal integrada de VS Code:

```
gcc -Wall -Wextra -std=c11 -o programa main.c global.c
./programa
```

2. Primera Parte — Librería global.h

En esta sección se deberá implementar una librería llamada `global.h` junto con su archivo de implementación `global.c`. La librería debe incluir internamente todos los headers necesarios para su funcionamiento. Todos los tipos personalizados también deben estar definidos en esta librería.

> ⚠ **Restricción de nombres:** Los nombres de funciones deben ser exactamente los especificados. Cualquier variación en el nombre o firma implicará la corrección del ítem como incorrecto.

2.1 Tipos Definidos (typedef / struct)

Para garantizar la compatibilidad con el testbench automático, los tipos deben definirse **exactamente** como se indica a continuación.

coeff_t — Coeficientes de ecuación cuadrática ($ax^2 + bx + c = 0$)

Tipo	Campo	Descripción
`int32_t`	`a`	Coeficiente cuadrático
`int32_t`	`b`	Coeficiente lineal
`int32_t`	`c`	Término independiente

root_t — Raíces de la ecuación cuadrática

Tipo	Campo	Descripción
`int32_t`	`real1`	Parte real de la primera raíz
`int32_t`	`imag1`	Parte imaginaria de la primera raíz (0 si es real)
`int32_t`	`real2`	Parte real de la segunda raíz
`int32_t`	`imag2`	Parte imaginaria de la segunda raíz (0 si es real)
`bool`	`complex`	true si las raíces son complejas conjugadas

complex_t — Número complejo

Tipo	Campo	Descripción
`int32_t`	`real`	Parte real
`int32_t`	`imag`	Parte imaginaria

date_t — Fecha

Tipo	Campo	Descripción
`int8_t`	`day`	Día (1–31)
`int8_t`	`month`	Mes (1–12)
`int16_t`	`year`	Año (ej.: 2025)

matriz_t — Matriz dinámica

Tipo	Campo	Descripción
`int16_t **`	`data`	Puntero a la matriz de valores (filas × columnas)
`size_t`	`rows`	Número de filas
`size_t`	`cols`	Número de columnas

Nota: Para cada tipo nuevo se deberá implementar una función `print_<tipo>(...)` que muestre todos sus atributos en pantalla de forma legible.

2.2 Funciones Requeridas

La siguiente tabla resume todas las funciones que deben implementarse en global.h / global.c.

Retorno	Función	Parámetros	Descripción
`void`	`init_lab`	`void`	Imprime en pantalla los nombres de los integrantes del grupo.
`root_t *`	`eq_solver`	`coeff_t *coeficientes`	Resuelve la ecuación cuadrática $ax^2+bx+c=0$. Admite raíces complejas.
`int32_t`	`bin2dec`	`char *binary,` `bool sign`	Convierte un número binario (en string) a decimal. *sign* = true si es con signo.

`void`	`print_reverse_array`	`void *array,`
		`size_t data_type,`
		`size_t array_size`
			Imprime el array en orden inverso usando puntero genérico.

`void`	`max_index`	`void *array,`
		`size_t data_type,`
		`size_t array_size`
			Imprime el índice y valor máximo del array.

`void`	`min_index`	`void *array,`
		`size_t data_type,`
		`size_t array_size`
			Imprime el índice y valor mínimo del array.

Retorno	Función	Parámetros	Descripción
`matriz_t *`	`matrix_sub`	`matriz_t A,` `matriz_t B`	Retorna la resta A − B. Las matrices deben tener las mismas dimensiones.

`int`	`swap`	`void *elem_1,`
		`void *elem_2,`
		`size_t data_type`
			Intercambia dos elementos.
			Retorna 0 si tuvo éxito, -1 si falló.

`int`	`consonantes`	`char *string`	Cuenta y retorna el número de consonantes en el string.
`int`	`vocales`	`char *string`	Cuenta y retorna el número de vocales en el string.
`char *`	`reverse_string`	`char *string`	Imprime y retorna una copia del string al revés.
`int32_t`	`string_length`	`char *string`	Longitud del string sin usar funciones de biblioteca. Retorna -1 si error.
`int32_t`	`string_words`	`char *string`	Cuenta palabras del string sin usar librerías estándar.
`int`	`string_copy`	`char *source,` `char *destination`	Copia *source* en *destination* sin usar librerías estándar.
`int`	`find_in_string`	`char *haystack,` `char *needle`	Busca *needle* en *haystack*. Retorna índice o -1 si no se encuentra.
`void`	`string_to_caps`	`char *string`	Convierte el string a mayúsculas (sin librerías estándar, in-place).
`void`	`string_to_min`	`char *string`	Convierte el string a minúsculas (sin librerías estándar, in-place).
`complex_t *`	`sum`	`complex_t a,` `complex_t b`	Retorna la suma de dos números complejos.
`complex_t *`	`prod`	`complex_t a,` `complex_t b`	Retorna el producto de dos números complejos.
`int`	`days_left`	`date_t start,` `date_t finish`	Calcula la diferencia en días entre dos fechas.

2.3 Notas de Implementación

- **Funciones de string:** Las funciones marcadas "sin librerías estándar" no pueden usar strcmp, strlen, strcpy, strstr ni similares de <string.h>.
- **Punteros genéricos:** Las funciones que reciben void* deben usar el parámetro data_type (sizeof del tipo) para aritmética de punteros.
- **Memoria dinámica:** Las funciones que retornan punteros deben usar malloc/calloc. El llamador es responsable de liberar la memoria.
- **Ecuación cuadrática:** Manejar correctamente los casos discriminante > 0, = 0 y < 0 (raíces complejas conjugadas).
- **Fechas:** La función days_left debe considerar correctamente los años bisiestos.

3. Segunda Parte — Lista Enlazada de Estudiantes

En esta sección se deberá implementar una **lista enlazada simple** para gestionar datos de estudiantes. La implementación debe estar dividida en los archivos `estudiantes.h` y `estudiantes.c`, más un programa principal `main.c`.

3.1 Datos del Estudiante

Cada nodo de la lista debe almacenar los siguientes campos:

estudiante_t — Datos del estudiante

Tipo	Campo	Descripción
`char [64]`	`nombre`	Nombre del estudiante
`char [64]`	`apellido`	Apellido del estudiante
`uint32_t`	`ci`	Cédula de identidad (número único)
`uint8_t`	`grado`	Año o grado de cursada
`float`	`promedio`	Promedio de calificación (0.0 – 100.0)

3.2 Operaciones Requeridas

- Agregar un estudiante a la lista (inserción dinámica).
- Eliminar un estudiante de la lista por CI.
- Mostrar toda la lista en pantalla.
- Mostrar un estudiante determinado (búsqueda por CI o nombre).
- **Ordenar y mostrar la lista por apellido (orden alfabético ascendente).**
- **Ordenar y mostrar la lista por CI (orden numérico ascendente).**
- Mostrar la letra de calificación correspondiente al promedio de cada estudiante.

3.3 Escala de Calificaciones

Rango	Letra	Descripción
0 – 30	**D**	Deficiente
31 – 60	**R**	Regular
61 – 75	**B**	Bueno
76 – 81	**BMB**	Bueno Muy Bueno
82 – 94	**MB**	Muy Bueno
95 – 100	**S**	Sobresaliente

3.4 Requerimientos Generales

- **Código comentado:** Cada función debe tener un comentario de cabecera explicando su propósito, parámetros y valor de retorno.
- **Estilo y estructura:** Usar typedef, #define, indentación consistente (4 espacios o 1 tab), y separar declaraciones de implementaciones.
- **Librería propia:** No mezclar la implementación en el main. Toda la lógica de la lista debe estar en estudiantes.h / estudiantes.c.
- **Tope de líneas:** El proyecto completo no debe superar las 1200 líneas de código (sin contar líneas en blanco ni comentarios).
- **Sin memoria leída:** Toda memoria dinámica reservada con malloc/calloc debe ser liberada antes de terminar el programa.

4. Criterios de Evaluación

Criterio	Detalle
Nombres y firmas de funciones correctos	Obligatorio. Errores implican ítem incorrecto.
Correcto funcionamiento de cada función	Verificado con testbench automático.
Estructura del archivo entregado	Carpetas y nombres de archivos según especificación.
Calidad del código (comentarios, estilo)	Claridad, indentación, uso de typedef/define.
Manejo de memoria dinámica	Sin leaks; toda la memoria debe liberarse.
Presentación oral (si aplica)	Explicación del código y decisiones de diseño.

⚠ **Plagio:** La presentación de código copiado de otro estudiante o generado íntegramente por IA sin comprensión propia resultará en la anulación del laboratorio para todos los involucrados.